System Design Notes: Reliability & Operations

Nov 5, 2026 · @Salike Hassan

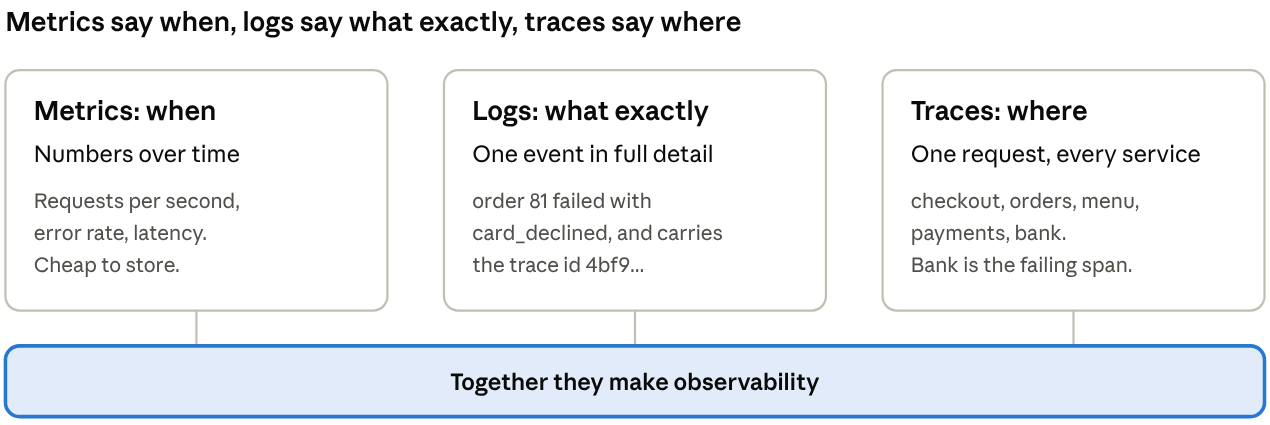
Observability for distributed systems

Observability matters in any modern app, distributed or a modular monolith. It is built from **three signals**: metrics, logs and traces. The order doesn't matter, what matters is that all three work together.

- **Metrics:** numbers over time, like requests per second, error rate, latency. Cheap to store, and great for spotting that something changed.
- **Logs:** the detailed record of one event, so we know exactly what happened.
- **Traces:** the path of one request across services, so we know where it happened.

Why metrics come first: with a good metrics setup, the moment latency or errors spike, the developer or stakeholder gets an alert on time. Without it, the app can be down for a long time and no one realizes. Logs and traces are what we open after the alert to find where and what.

Three signals at a glance



three signals · when, what exactly, where

The checkout example: metrics tell us the error rate went up, the trace shows the bank call failing, and the log says order 81 hit card_declined.

Metrics: golden signals, RED and USE

Google's SRE book names **four golden signals**. RED and USE are two shorthands built on them: **RED is for services, USE is for resources**.

Four golden signals (checkout service)

- **Latency:** how long a request takes. Checkout taking 4 seconds instead of 400 ms.
- **Traffic:** how much demand there is. Checkout requests per second.
- **Errors:** how many requests fail. Failed checkouts (5xx).
- **Saturation:** how full the service is. Threads, connection pool or queue close to the limit.

RED vs USE

RED (the service view): how many requests come in (rate), how many fail (errors), how long they take (duration). This is what the user feels.

USE (the resource view): one check for each CPU, disk, memory, connection pool.

- **Utilization:** how much of the resource is in use. CPU at 70% busy.
- **Saturation:** how much work is waiting because the resource is full. Requests queued for a DB connection.
- **Errors:** errors the resource itself reports. Disk read failures, dropped packets.

Utilization and saturation are not the same. A CPU at 100% with nothing waiting is busy but coping. Once a queue builds up, it is saturated and latency starts to climb.

How they work together: RED on checkout tells us users are hurting (duration is up). USE on its DB and CPU tells us why (the connection pool is saturated).

Percentiles: P90 and P99

P90 and P99 are the way we read **latency** (the Duration in RED). Sort all request times from fastest to slowest:

- **P90:** 90% of requests finished at or under this time. 1 in 10 was slower.
- **P99:** 99% of requests finished at or under this time. 1 in 100 was slower.
- **P50 (median):** the typical request.

Why not the average: the average hides the slow ones. Take 100 checkout requests: 98 take 100 ms and 2 take 5 seconds.

- Average is about 198 ms, which looks healthy.
- P90 is 100 ms.
- P99 is 5 seconds, so those 2 users are having a bad time.

The slow tail is also where the unhappy users, timeouts and retries live. So we alert on P99, not on the average.

How to derive it

1. Collect the duration of every request.
2. Sort them from fastest to slowest.
3. Take the value at position `p% x N`. With 1,000 requests, P99 is the 990th value.

Simple example with 10 requests (ms): 120, 95, 3100, 110, 105, 130, 98, 102, 115, 90.

1. Sorted: 90, 95, 98, 102, 105, 110, 115, 120, 130, 3100.
2. **P50:** 50% x 10 = position 5, so 105 ms.
3. **P90:** 90% x 10 = position 9, so 130 ms. 9 of the 10 requests were at or under 130 ms.
4. **P99:** 99% x 10 = 9.9, round up to position 10, so 3100 ms. With only 10 requests, the P99 is the slowest one.

The position is always rounded up. With 1,000 requests, P99 is position 990: 990 requests are at or under that value and 10 are slower.

We don't store every request in a metrics system. Instead the app records durations into **histogram buckets** (counts of requests under 50 ms, 100 ms, 250 ms...), and the metrics tool estimates the percentile from the buckets. In Prometheus: `histogram_quantile(0.99, ...)`.

If the durations are in a SQL Server table, we can calculate it directly:

```
SELECT DISTINCT PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY duration_ms)
OVER () AS p99_ms
FROM request_log;
```

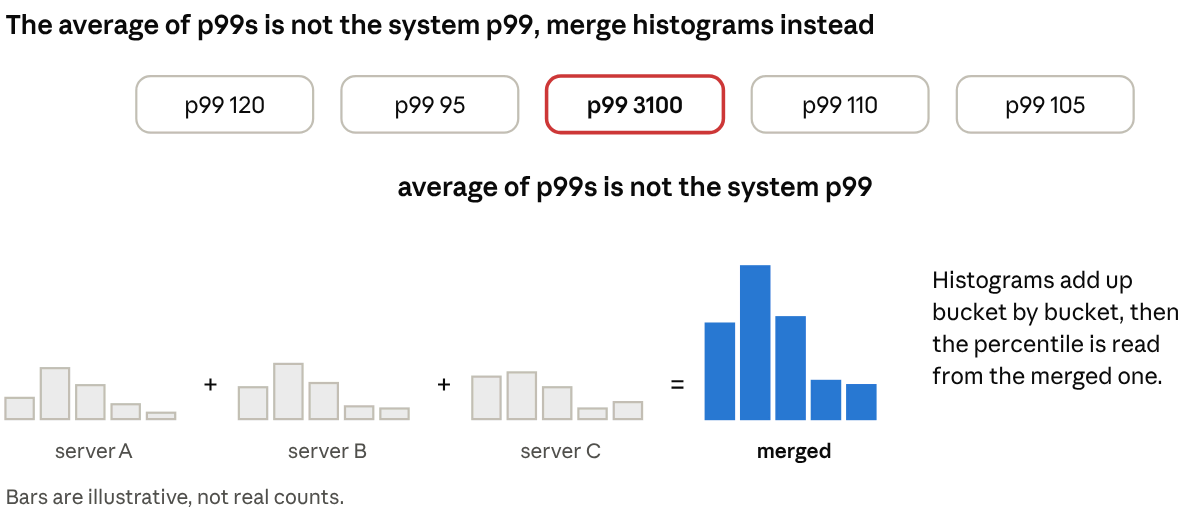
Trap: don't average the P99 of each server to get the overall P99. Merge the histograms from all servers first, then calculate the percentile on the merged data.

Two metric traps

1. Averaging p99s

Five servers report a p99 of 120, 95, 3100, 110 and 105 ms. Averaging them gives about 706 ms, which matches nothing real: four servers are fine and one is very slow. **Average of p99s is not the system p99.**

- **Fix:** record **histograms**, which are counts of requests in latency buckets.
- Histograms from every server can be **added up** bucket by bucket. The percentile is then calculated on the merged histogram, later.



p99 per server vs merged histograms

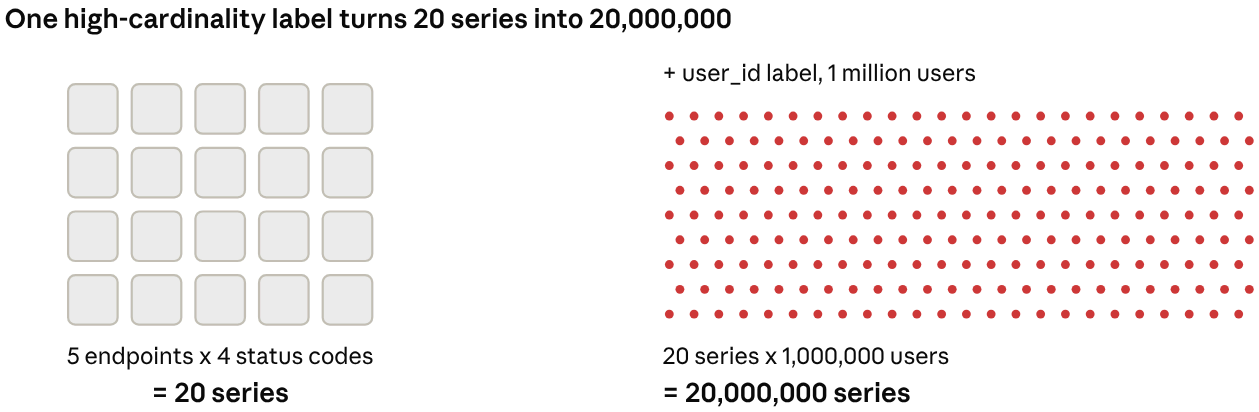
2. Cardinality

Cardinality is the number of distinct values a label can take.

A metric is a name plus labels, for example `http_requests_total{endpoint="/checkout", status="200"}`. Every **unique combination of label values is its own time series**, and each series is stored, indexed and billed separately.

- **Low cardinality:** endpoint (5 values), status code (4 values). Safe to use as labels.
- **High cardinality:** `user_id` (1 million values), `order_id`, `trace_id`, email. Almost every value is new, so each one creates a new series.

The number of series is the labels' cardinalities multiplied together: 5 x 4 = 20. Add `user_id` and it becomes 20 x 1,000,000 = **20,000,000 series**. That is how metrics bills explode.



Fix: keep labels low-cardinality. Per-user detail belongs in logs and traces.

cardinality · 20 series vs 20,000,000 series

- **20 million series is not 20 million requests.** It is 20 million separate time series to store and query, whatever the traffic is.
- **Sampling doesn't fix it.** Sampling is for traces and logs. For metrics the control is cardinality: choose labels carefully and set **cardinality limits**.
- **Fix:** keep metric labels low-cardinality (endpoint, status code). Per-user detail belongs in **logs and traces**, where each event is stored on its own and can carry `user_id`.

Structured logs

A plain text log like `payment failed for order 81 :(` is hard to search. A **structured log** writes the same event as fields (JSON), so we can filter on them:

```
{ "event": "payment_failed",
  "order_id": "81", "customer": "c_42",
  "error": "bank_timeout", "latency_ms": 3412,
  "bank": "acme", "trace_id": "4bf92f35..." }
```

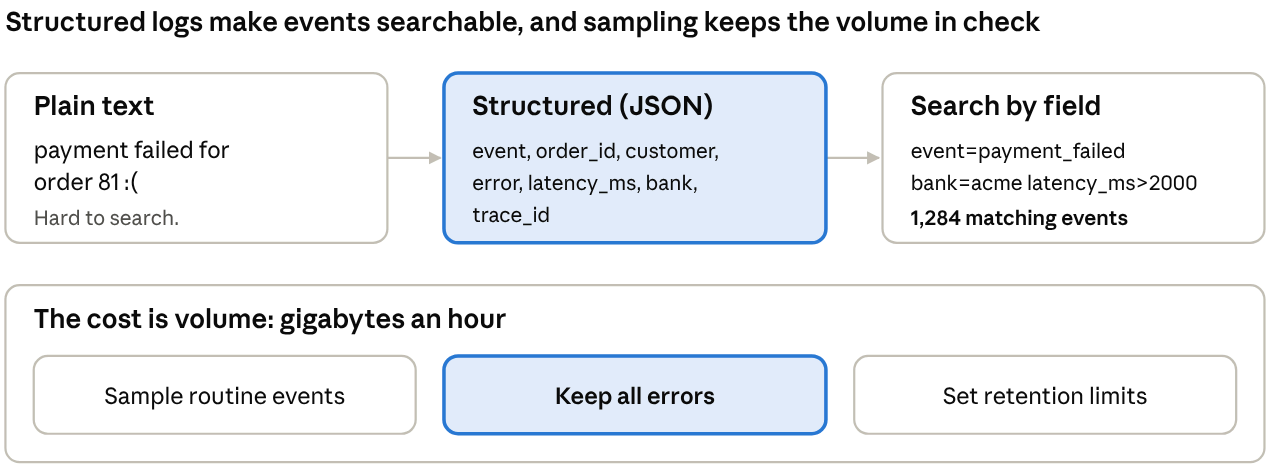
Now we can search: every failed payment in the last hour, for one bank, slower than 2 seconds.

```
event=payment_failed bank=acme latency_ms>2000 last=1h
-> 1,284 matching events
```

The `trace_id` field is what links a log line to its trace.

The cost is volume. A busy service can write gigabytes an hour. So teams:

- **Sample routine events:** keep only a share of the boring ones.
- **Keep all errors:** errors are stored in full.
- **Set retention limits:** logs are deleted after a fixed time.

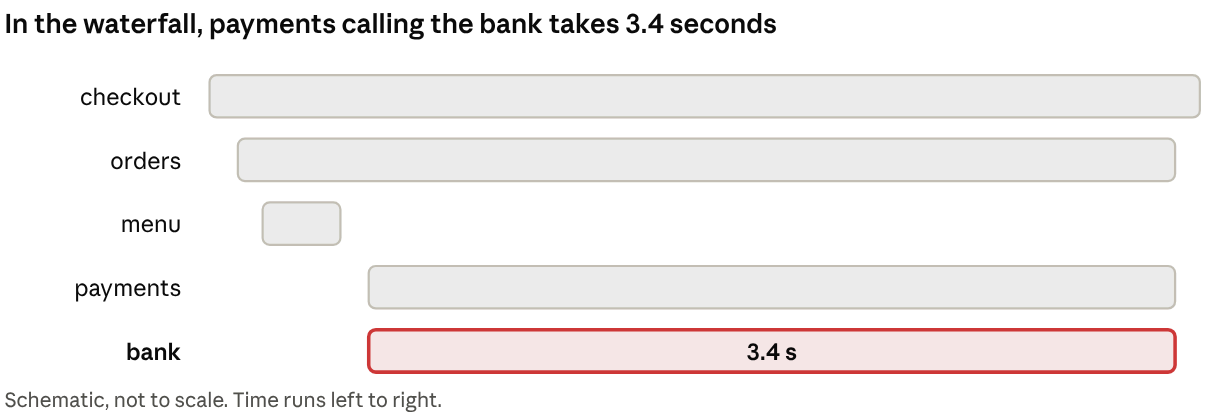


plain text vs structured logs · search and volume levers

Distributed tracing

When a checkout request enters the system, it gets a **trace id**. Every service that handles it records a **span**: what it did, when it started, and how long it took.

- The trace id is **propagated** from service to service with the request. That is how the spans from different services join into one trace.
- Laid out on a timeline, the tree of services becomes a **waterfall**, and the slow part jumps out.

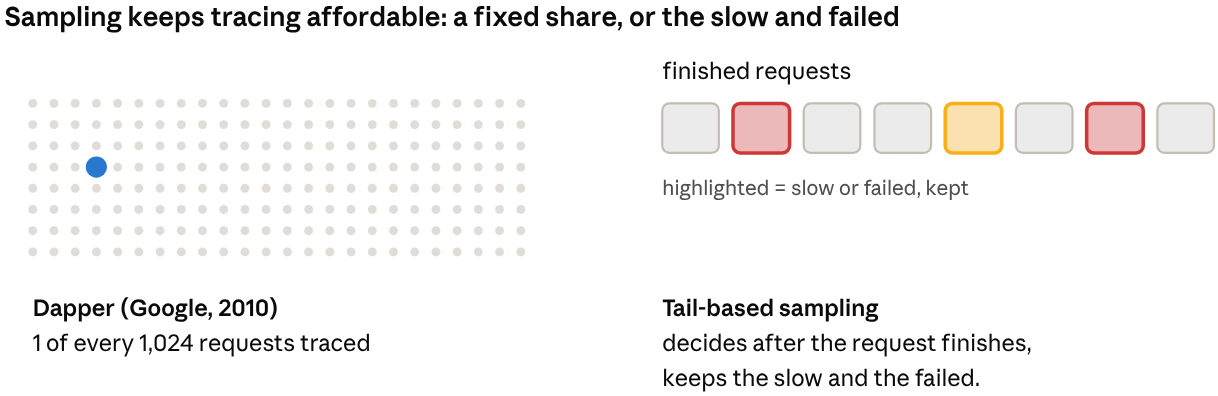


trace waterfall · checkout, orders, menu, payments, bank

Sampling

Tracing every request is costly, so we sample.

- **Dapper (Google, 2010)** made tracing practical at scale. The first version traced just 1 request in every 1,024, and that was still enough for most questions.
- **Tail-based sampling:** modern collectors decide **after the request finishes**, and keep all the slow and failed ones.



Dapper sampling vs tail-based sampling

Why sampling is enough: most requests look alike, so a small share shows the normal path. The risk with a fixed share is missing the rare slow or failed request, which is exactly what we need during an incident. That is why tail-based sampling keeps all the slow and failed ones.

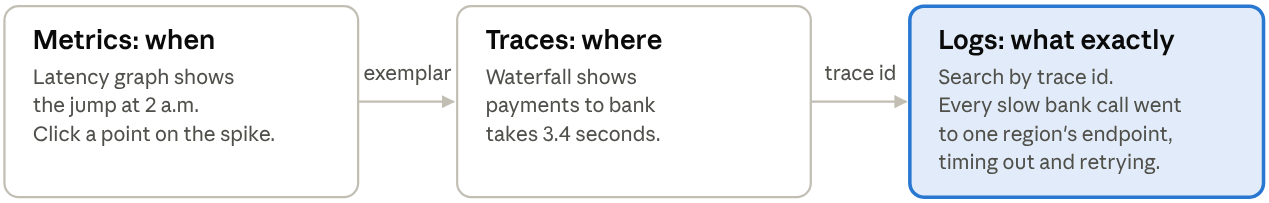
Trade-off: the collector has to hold all the spans of a request until it finishes before it can decide, so tail-based sampling costs more than a fixed share.

Sampling vs metrics: sampling is for traces and logs. Metrics are aggregates (counts and histograms) of every request, so they still show *when* something changed.

Solving the 2 a.m. page

This is where the three signals are linked together.

From the latency spike to the cause: metric, trace, logs



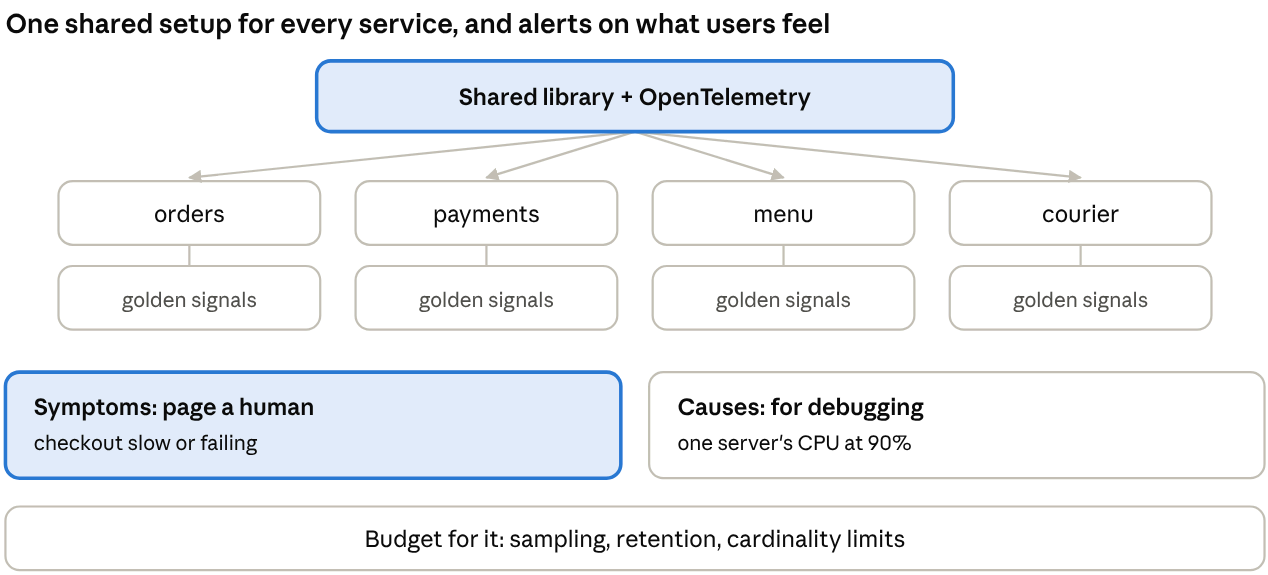
2 a.m. page · metric to trace to logs

1. **Metric:** the latency graph shows the jump. Click a point on the spike and it links to an example trace from that moment. That link is called an **exemplar**.
2. **Trace:** the waterfall shows payments calling the bank takes 3.4 seconds.
3. **Logs:** take the trace id and search the logs. Every slow bank call went to one region's endpoint, which is timing out and retrying.

```
trace=4bf9 bank=acme endpoint=eu-west-2 timeout retry=1
trace=4bf9 bank=acme endpoint=eu-west-2 timeout retry=2
trace=7c21 bank=acme endpoint=eu-west-2 timeout retry=1
```

Observability habits

- **One shared library plus OpenTelemetry:** every service (orders, payments, menu, courier) emits its signals the same way.
- **Basic golden-signals dashboard per service,** named the same way everywhere.
- **Alert on symptoms, not causes:** page a human for what users feel (checkout slow or failing), not for every cause (one server's CPU at 90%). Causes are for debugging, symptoms are for waking people up.
- **Budget for it:** sampling, retention and cardinality limits. Observability data can grow faster than the system it observes.



observability habits · shared setup, symptoms vs causes, budget

Recap

1. Metrics: when. Traces: where. Logs: what.
2. Golden signals, histograms, low-cardinality metrics.
3. Propagate trace ids, keep the slow and failed traces.
4. Link the signals, alert on what users feel.

Demo: Azure Application Insights and Datadog

The goal is to reproduce the 2 a.m. page with a small app, and show metrics, traces and logs linked together in either tool.

Demo app (same for both tools)

- Three small ASP.NET Core APIs: **checkout, orders, payments**. Payments calls a fake **bank** endpoint.
- A switch on the bank stub (an environment variable) that adds a delay and a failure rate, for example `BANK_DELAY_MS=3400`. That makes payments to bank the slow span, like the waterfall.
- Instrument with **OpenTelemetry**, so the same app can point at either tool.
- Write **structured JSON logs** with fields (`event`, `order_id`, `bank`, `latency_ms`, `error`), and keep the trace id on every log line.

- A traffic loop (a simple script or k6): normal traffic for a few minutes, then flip the bank switch.

Option 1: Azure Application Insights

1. Create an Application Insights resource and copy its connection string. Set it as the environment variable `APPLICATIONINSIGHTS_CONNECTION_STRING`.
2. Add the package and one line of startup code. Microsoft's current docs recommend the Microsoft OpenTelemetry Distro for new apps. The earlier `Azure.Monitor.OpenTelemetry.AspNetCore` package with `UseAzureMonitor()` still works.
3. Name each service with `OTEL_SERVICE_NAME` (the Cloud Role Name), so checkout, orders and payments show up as separate nodes in the Application Map.

```
// dotnet add package Microsoft.OpenTelemetry
using Microsoft.OpenTelemetry;

var builder = WebApplication.CreateBuilder(args);

builder.UseMicrosoftOpenTelemetry(options =>
{
    options.Exporters = ExportTarget.AzureMonitor;

    // Demo: keep every trace so nothing is missing
    options.AzureMonitor.TracesPerSecond = null;
    options.AzureMonitor.SamplingRatio = 1.0F;
});
```

4. **Sampling demo:** the distro samples at about 5 traces per second by default. Drop `SamplingRatio` to `0.1F` (about 10% of traces) and show traces going missing. Metrics are never sampled, so alert on metrics.
5. **Find the signals in the portal:** metrics on the Overview and Metrics charts, traces in the Application Map and the end-to-end transaction view (the waterfall), logs in the Logs blade with KQL.
6. **Alert on the symptom:** an alert rule on failed requests or checkout duration, not on CPU.

KQL for the P90 and P99 of the bank call:

```
dependencies
| where timestamp > ago(1h) and name has "bank"
| summarize p90 = percentile(duration, 90), p99 = percentile(duration, 99) by
bin(timestamp, 5m)
| render timechart
```

KQL for the logs of one trace (`operation_Id` is the trace id):

```
traces
| where operation_Id == "<trace id from the transaction view>"
| project timestamp, message, customDimensions
```

Option 2: Datadog

1. Install the Datadog Agent. On Azure App Service, use the Datadog extension for Azure App Service, which collects traces and custom metrics for .NET apps.
2. Add the Datadog .NET tracer (automatic instrumentation) and set `DD_ENV`, `DD_SERVICE` and `DD_VERSION`.
3. **Logs:** send them as JSON. Trace correlation is on by default from tracer version 3.24.0 (older versions need `DD_LOGS_INJECTION=true`), and the logs get `dd_trace_id` and `dd_span_id`. With Microsoft.Extensions.Logging, turn on log scopes and the JSON formatter.
4. **Find the signals:** a dashboard and a monitor for checkout latency and errors, the APM trace view for the waterfall, and the Log Explorer with a search like `service:payments @latency_ms:>2000`.
5. **Sampling demo:** traces and logs are sampled independently, so a log can carry a trace id whose trace was not kept. That is not a misconfiguration, it is the sampling lesson.
6. **Cardinality demo:** Datadog bills custom metrics by unique tag combinations, so adding a `user_id` tag to a metric is the live version of the 20,000,000 series example.

Where each signal lives

Signal	Application Insights	Datadog
Metrics (when)	Overview and Metrics charts, alert rules	Dashboards and monitors

Signal	Application Insights	Datadog
Traces (where)	Application Map, end-to-end transaction view	APM trace view
Logs (what exactly)	Logs blade (KQL), `operation_Id` = trace id	Log Explorer, logs carry `dd_trace_id`
Sampling	Rate-limited by default, or fixed percentage. Metrics never sampled	Ingestion controls. Traces and logs sampled independently

Demo flow

1. Run normal traffic: the golden-signals dashboard is flat and P99 is healthy.
2. Flip the bank switch: the latency graph jumps and the alert on the symptom fires (**metrics: when**).
3. Open a sample trace from the slow requests: the waterfall shows payments to bank at about 3.4 seconds (**traces: where**).
4. Search the logs by that trace id: timeouts and retries on one endpoint (**logs: what exactly**).
5. Show the cost levers: lower the sampling ratio, then add a `user_id` metric label and watch the series count.

Sources

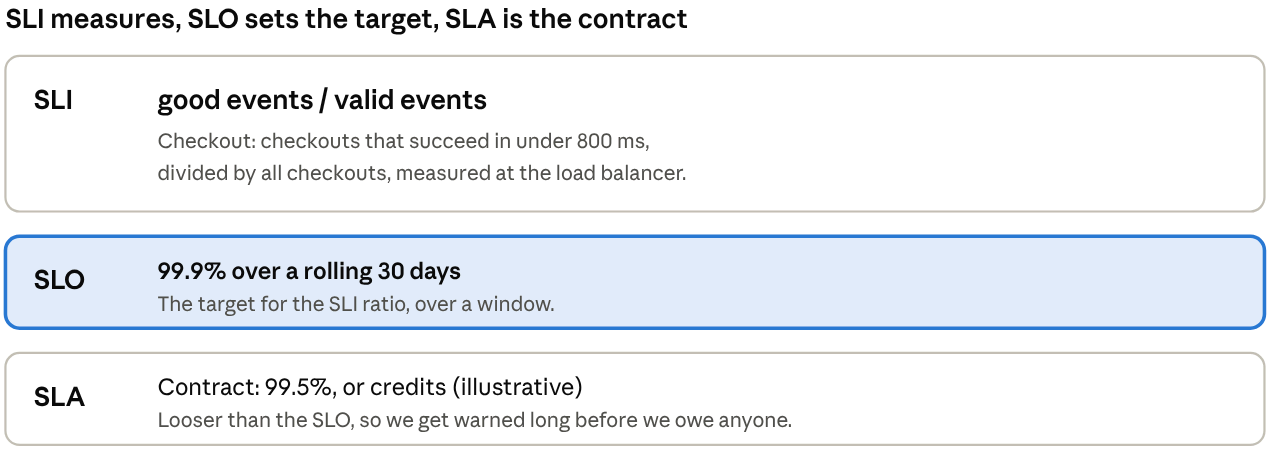
- Azure Monitor OpenTelemetry: enable (Microsoft Learn source)
- Configure OpenTelemetry in Application Insights: sampling, Cloud Role Name
- Datadog: correlating .NET logs and traces
- Datadog: correlate logs and traces

SLI, SLO and SLA

Reliability has to be defined as a number, so we can agree on it, track it and decide with it.

- **SLI (service level indicator):** the measurement. `good events / valid events`. For checkout: the checkouts that succeed in under 800 ms, divided by all checkouts, measured at the load balancer, as close to the user as we can get.
- **SLO (service level objective):** the target for that ratio over a window. 99.9% over a rolling 30 days.

- **SLA (service level agreement):** a contract with customers, with money attached if it's broken (credits). It is usually looser than the SLO (for example 99.5%, illustrative), so we get warned long before we owe anyone.



SLI, SLO and SLA

What is a good event?

Correction on the P99 question: a good event is not "a checkout under the P99". It is one request that meets the bar we chose: it succeeds **and** finishes within 800 ms.

- **Valid events** are the requests that count toward the SLI. For example, leave out health checks and bad client requests.

Worked calculation: SLI, SLO and SLA

Checkout, last 30 days, **1,284,716 requests** in total. A good event is a request that completed **under 800 ms**. (A failed request would also count as bad. It is left out here to keep the example simple.)

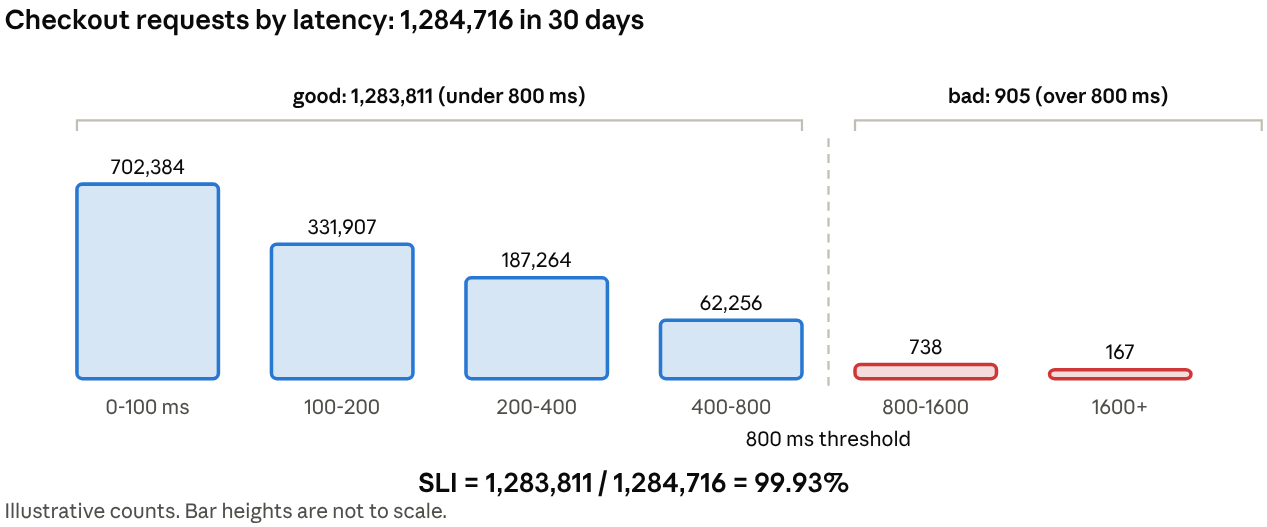
Step 1: count the requests per latency bucket (the histogram).

Latency	Requests
0-100 ms	702,384
100-200 ms	331,907
200-400 ms	187,264
400-800 ms	62,256

Latency	Requests
800-1600 ms	738
over 1600 ms	167
Total	**1,284,716**

Step 2: pick the valid and good events.

- **Valid events** = all the requests = **1,284,716**.
- **Good events** = requests that completed under 800 ms = the first four rows: 702,384 + 331,907 + 187,264 + 62,256 = **1,283,811**.



checkout requests by latency and the SLI

Step 3: the SLI. SLI = good / valid = 1,283,811 / 1,284,716 = **99.93%**.

Step 4: the SLO. Target: 99.9% over a rolling 30 days.

- Our SLI is 99.93%, which is above 99.9%, so the **SLO is met**.
- Allowed bad requests = 0.1% x 1,284,716 = **1,284.7**. We had 905, so we are within it.

Why: the SLO turns "reliable enough" into one agreed number, so the team knows when it is safe to ship and when to stop and fix.

Step 5: the SLA. Contract: 99.5% (illustrative).

- Our SLI is 99.93%, which is above 99.5%, so the **SLA is met**.
- Allowed bad requests = 0.5% x 1,284,716 = **6,423.6**. We had 905.

Why: the SLA is the promise we make to customers, set looser than the SLO so that missing the SLO warns us long before we owe anyone anything.

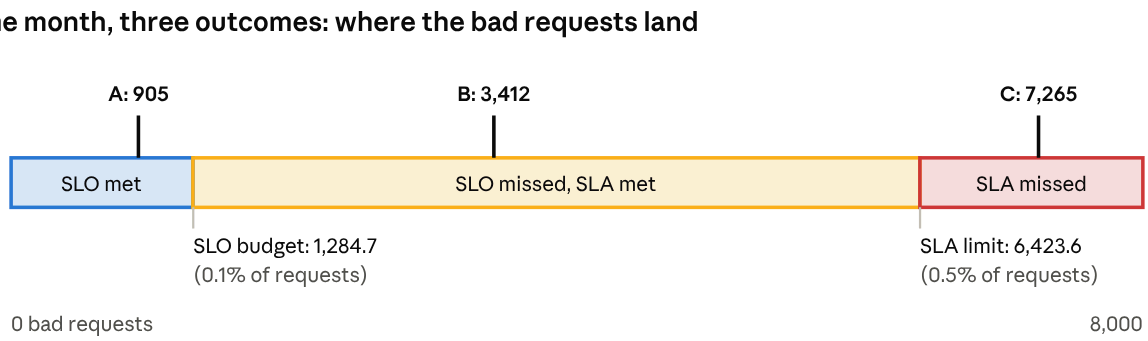
In Application Insights (KQL):

```
requests
| where timestamp > ago(30d) and name has "checkout"
| summarize good = countif(success == true and duration <= 800), valid =
count()
| extend sli_pct = 100.0 * good / valid
```

Same month, three outcomes: SLO, SLA and the error budget

Same setup: 1,284,716 requests, SLO 99.9% (error budget = 0.1% = **1,284.7** bad requests), SLA 99.5% (limit = 0.5% = **6,423.6** bad requests). Only the number of bad requests changes.

Scenario	Bad	Good	SLI	SLO 99.9%	Error budget used	SLA 99.5%
A: good month (the example above)	905	1,283,811	99.93%	Met	70.4%	Met
B: bad month	3,412	1,281,304	99.73%	Missed	265.6%	Met
C: very bad month	7,265	1,277,451	99.43%	Missed	565.5%	Missed



bad requests against the SLO budget and the SLA limit

Scenario B, step by step:

- Good = 1,284,716 - 3,412 = 1,281,304. SLI = 1,281,304 / 1,284,716 = **99.73%**.
- 99.73% is below 99.9%, so the **SLO is missed**.

- Error budget: 1,284.7 allowed, 3,412 used. 3,412 / 1,284.7 = 265.6%, so the budget is spent, and then some.
- 99.73% is above 99.5% (3,412 is below the 6,423.6 limit), so the **SLA is still met**. No credits.

Scenario C: SLI = 1,277,451 / 1,284,716 = **99.43%**. That is below 99.5%, so the **SLA is missed** (7,265 is 841.4 over the 6,423.6 limit). Credits are owed.

What we do with the error budget:

- **A, 29.6% left:** keep shipping features.
- **B, budget gone:** the error budget policy kicks in. Feature launches pause and the team works on reliability until the budget recovers. We missed our own target, but we owe customers nothing.
- **C, budget gone and SLA missed:** the same policy, plus credits to customers and a blameless postmortem.
- **The warning zone:** the budget runs out at 1,284.7 bad requests, long before the SLA limit at 6,423.6. That gap is the warning, so we react while the contract is still safe.

Are the SLO and SLA calculated or hypothetical?

Neither. They are decided. Only the SLI is calculated, from real data. The 99.9% and 99.5% in the examples are illustrative picks. In a real project the team and the business choose them.

How to decide the SLO

1. Measure the SLI for 2 to 4 weeks to get a baseline. For example, 99.93%.
2. Ask what the business and users need: how much does a slow or failed checkout hurt?
3. Pick a target we can defend: close to what we achieve today, never above reality, and never 100%.
4. Check the cost. Each extra nine costs roughly 10x more, and the dependencies we call cap what we can promise.
5. Agree it with product and engineering. Review it every quarter, and tighten it when it is met with budget to spare.

How to decide the SLA

- It is a business and legal decision with the customer, not a formula.

- Start from the SLO and set it looser, with money attached (credits) that we can afford, and list the excluded causes.
- **Backtest it:** apply the candidate to past months. A good SLA would have been met in every one of them.

Backtest example (illustrative monthly SLIs):

Month	SLI	SLO 99.9%	SLA 99.5%
1	99.93%	Met	Met
2	99.88%	Missed	Met
3	99.95%	Met	Met
4	99.61%	Missed	Met
5	99.91%	Met	Met
6	99.97%	Met	Met

- The 99.9% SLO was met in 4 of 6 months. It is reachable, and it still has teeth when it is missed.
- The 99.5% SLA was met in all 6, so it is safe to put in a contract.
- A 99.95% SLO would have been met in only 2 of 6. That is too tight to start with.

Checkout alerts in real numbers (burn rate, without the theory)

The gap between 100% and the SLO is the **error budget**: the amount of badness we are allowed, and the thing we spend on shipping changes.

- **Budget** = 100% - 99.9% = **0.1%**
- **In checkouts:** 10,000,000 checkouts a month x 0.1% = **10,000 allowed bad** (10M a month is illustrative).
- **In time:** 43,200 minutes in a 30-day month x 0.1% = **43.2 minutes a month**.
- **One more nine:** at 99.99% the budget shrinks to 4.3 minutes a month. Barely time to notice a problem, let alone fix it.

Same checkout, SLO 99.9%. We want alerts in plain numbers, like a queue alert: X stuck messages for 5 minutes is a warning, X plus Y is an alert. Here X and Y are **bad checkouts in a 5-minute window**. A bad checkout is the same as in the SLI: it failed or took more than 800 ms.

Step 1: the traffic.

- 1,284,716 requests in 30 days / 30 = **42,824 a day**.
- 42,824 / 24 hours = **1,784 an hour**.
- 1,784 / 60 minutes = **30 a minute**.
- 30 x 5 minutes = **149 in a 5-minute window**.
- At 149 requests a window, one bad checkout is already 0.7% (1 / 149), so percentages jump around. That is why we count bad checkouts instead.

Step 2: the error budget.

- Budget = 100% - 99.9% = **0.1%**.
- Bad requests allowed in 30 days = 0.1% x 1,284,716 = **1,284.7**.
- 30 days = 30 x 24 = **720 hours**. So the allowed pace = 1,284.7 / 720 = **1.78 bad an hour** (about 1.8).
- A couple of bad checkouts now and then is normal. We only alert when the pace is far above 1.8 an hour.

Step 3: the warning number.

- Our choice: we don't want to lose more than about **5%** of the month's budget in an hour before a human looks.
- 5% of 1,284.7 = 1,284.7 x 0.05 = **64.2**. That is where the 64 comes from: 64 bad checkouts in an hour.
- We check in 5-minute windows, and an hour has 60 / 5 = **12 windows**. That is where the 12 comes from.
- Per window: 64.2 / 12 = 5.35, about **5**.
- **Warning: 5 or more bad checkouts in a 5-minute window.** If that kept up for the whole hour: 5 x 12 = 60 bad = 4.7% of the budget.

Step 4: the alert number.

- Our choice: about **15%** of the month's budget in an hour is too fast to wait.
- 15% of 1,284.7 = **192.7**, about 193 bad in an hour.
- 193 / 12 windows = 16.06, about **16**.
- **Alert: 16 or more bad checkouts in a 5-minute window.** If that kept up: 16 x 12 = 192 bad in an hour = 15% of the budget.

Where do the 5% and the 15% come from? There is no formula. They are our choice. We decide how long the budget should last if the problem keeps going, then work back to a percentage: **percent an hour = 100% / hours until the budget is gone**.

- **Warning:** if it keeps up, the budget should last about a day, so a human can look during working hours. 100% / 20 hours = **5% an hour**.
- **Alert:** if it keeps up, the budget would be gone within about one shift, so we can't wait for the morning. 100% / 8 hours = 12.5% an hour, and we rounded up to **15%**, which is gone in 6.7 hours.
- **Other teams choose differently.** Your earlier slide pages at 2% of the budget in an hour. That is 100% / 50 hours, the budget lasting about two days, and it is the 14.4x burn rate. A strict customer-facing product might pick 2%. A tolerant internal tool might pick more than 15%.

So the question to ask the team is: "if this keeps going, how many hours can we accept before the budget is gone?"

Step 5: how long the budget would last at those paces.

- Warning pace: 1,284.7 / 60 an hour = **21.4 hours**.
- Alert pace: 1,284.7 / 192 an hour = **6.7 hours**.

Step 6: the slower rules.

- **Slow leak:** 5% of the budget over 6 hours = 64.2, so **64 or more bad in 6 hours**. That is about 11 an hour: below the warning pace, but 6 times the allowed 1.78.
- **Ticket:** 10% of the budget over 3 days = 128.5, so **129 or more bad in 3 days**. 3 days is 72 hours, and 72 x 1.78 = 128.5, which is just the allowed pace. If we stay above it, the budget runs out before the month ends.

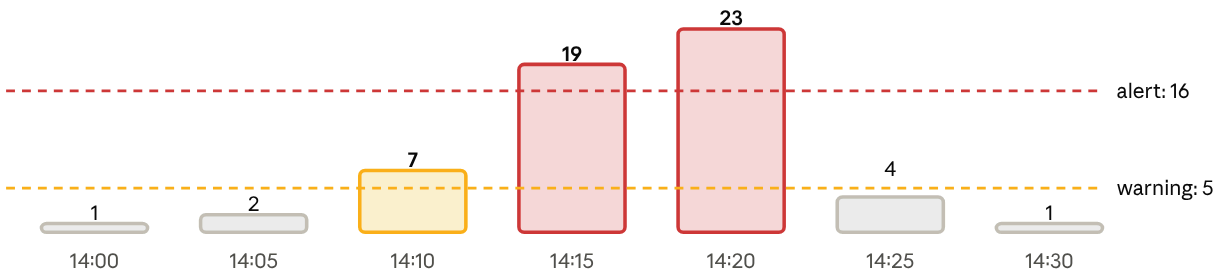
All the rules together (my example thresholds, the team chooses them):

Level	Rule for checkout	If it kept up	Who is told
Warning	5 or more bad checkouts in 5 minutes	60 an hour, 4.7% of the budget an hour, gone in about 21 hours	Message in the team channel
Alert	16 or more bad checkouts in 5 minutes	192 an hour, 15% of the budget an hour, gone in about 7 hours	Page the on-call

Level	Rule for checkout	If it kept up	Who is told
Slow leak	64 or more bad in 6 hours (5% of the budget)	A smaller problem that doesn't stop	Message in the team channel
Ticket	129 or more bad in 3 days (10% of the budget)	A slow burn	Open a ticket

Where burn rate comes in: burn rate is how many times faster than the allowed pace we are spending the budget. The allowed pace is 1.8 bad an hour. The warning pace of 60 an hour is about **34x** faster. The alert pace of 192 an hour is about **108x** faster. The bigger the number, the sooner the budget is gone, so the louder the alert.

Bad checkouts per 5-minute window during a bad deploy



Each bar is the 5-minute window ending at that time. Deploy at 14:06, rollback at 14:21.

Warning at 14:10, alert at 14:15. The incident used 53 bad checkouts, 4.1% of the month's budget.

checkout alerts in 5-minute windows

A bad deploy, minute by minute:

1. **14:06:** a deploy slows the payment calls.
2. **Window ending 14:10:** 7 bad checkouts. That is 5 or more, so a **warning** goes to the team channel.
3. **Window ending 14:15:** 19 bad. That is 16 or more, so the on-call is **paged**.
4. **14:21:** the on-call rolls the deploy back. Mitigate first, investigate later.
5. **Windows ending 14:25 and 14:30:** 4 bad, then 1. Back to normal.
6. **Cost:** 7 + 19 + 23 + 4 = 53 bad checkouts = 53 / 1,284.7 = **4.1% of the month's budget**. 95.9% is left, so features keep shipping and the error budget policy is not triggered.

A couple of bad checkouts now and then (the 1 and 2 at the start) is normal and stays quiet. That is why we don't alert on every failure.

The alert query in Application Insights (KQL), run every 5 minutes:

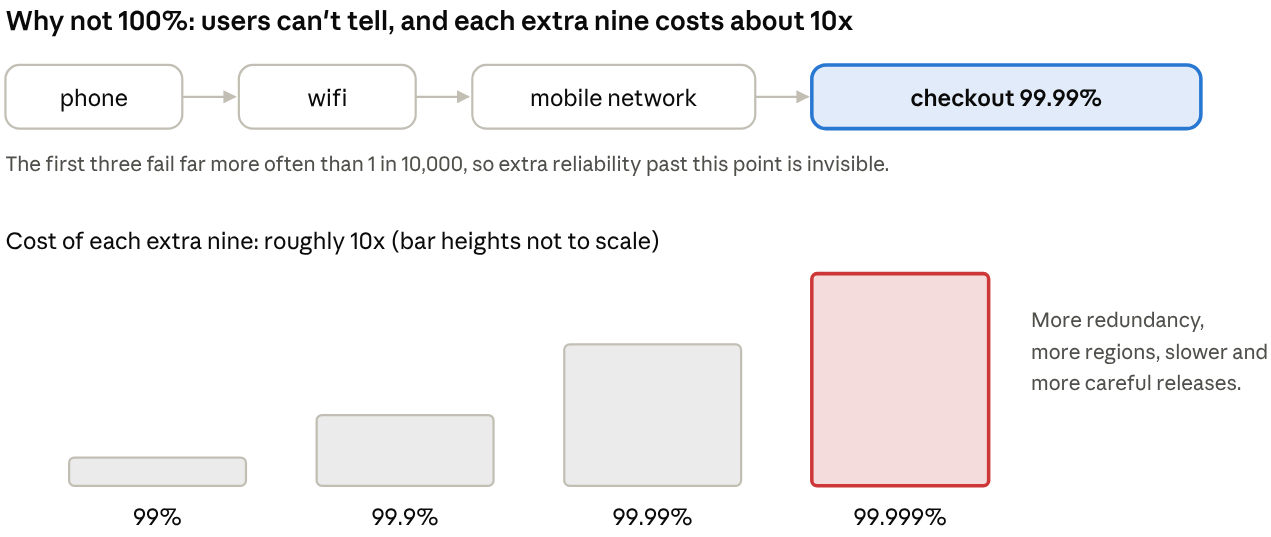
```
requests
| where timestamp > ago(5m) and name has "checkout"
| summarize bad = countif(success == false or duration > 800)
```

Warning when `bad >= 5`, alert when `bad >= 16`. A bad checkout here is the same definition as the SLI: it failed or took more than 800 ms.

When traffic grows, the counts stop making sense. At 10x the traffic, 5 bad in 5 minutes is just normal noise. Then switch to percentages (burn rate), or scale the counts with the traffic.

Why the target is never 100%

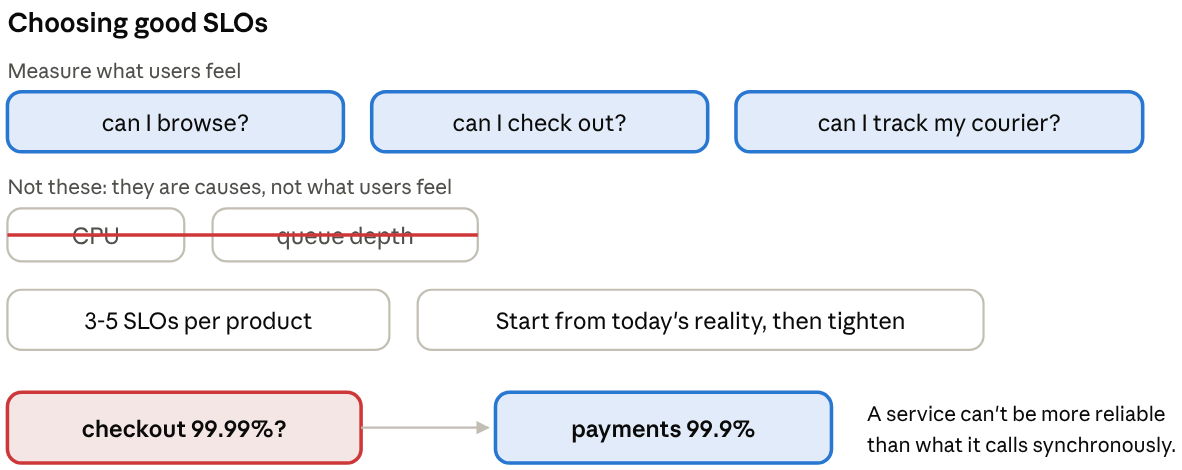
1. **Users can't tell.** Their phone, their wifi and their mobile network fail far more often than 1 request in 10,000. Above a point, extra reliability is invisible.
2. **Every extra nine costs roughly 10x more.** More redundancy, more regions, slower and more careful releases.
3. **100% means never deploy anything.** Every change is a risk. The Google SRE book (ch. 3, Embracing Risk) puts it as: 100% is the wrong reliability target for basically everything.



why not 100% · users can't tell, each nine costs 10x

Choosing good SLOs

- **Measure what users feel:** can I browse? Can I check out? Can I track my courier? Not CPU or queue depth, those are causes for debugging.
- **Keep it to 3-5 SLOs per product.**
- **Start from today's reality, then tighten.** An SLO we can't meet today only creates noise.
- **Mind the chain:** a service can't be more reliable than the dependencies it calls synchronously. If payments promises 99.9%, checkout can't promise 99.99%. In a serial chain the availabilities multiply, so checkout ends up a little below its weakest dependency.



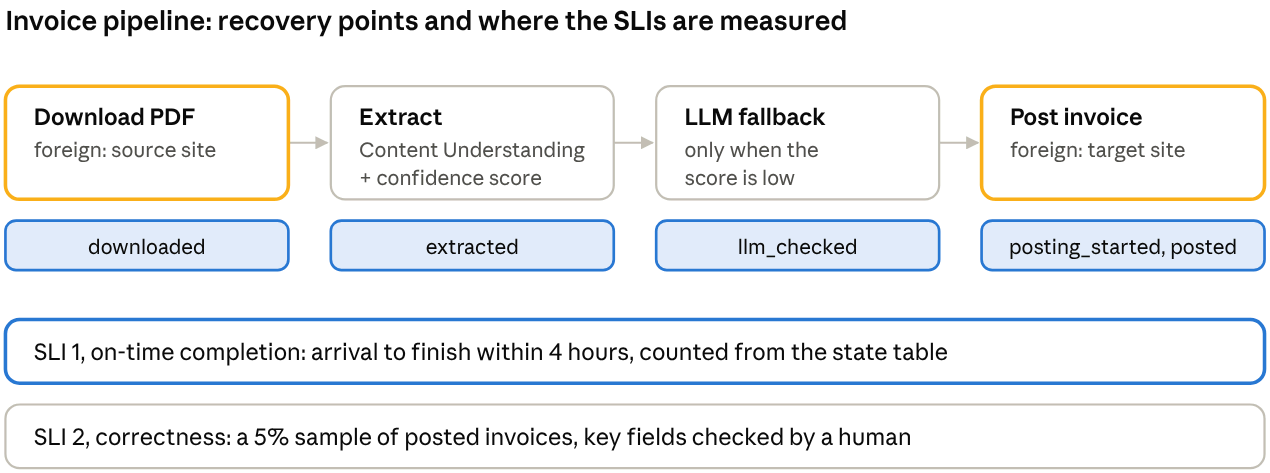
choosing good SLOs and the dependency chain

Recap

1. SLI = good / valid. SLO = the target. SLA = a looser contract.
2. Never 100%: 99.9% leaves 43 minutes a month.
3. Spend the budget on shipping, freeze when it's gone.
4. Alert on burn rate over multiple windows.

Worked example: SLI, SLO and error budget for invoice automation

The use case: read invoices from a source website, extract values with Content Understanding, fall back to an LLM when the confidence score is low, then post the data on a target website. Up to 1,000 invoices a day, so about 30,000 in 30 days. This example is built from your project description. The numbers are illustrative and the design choices are my suggestions, so calibrate them with real data.



invoice pipeline · recovery points and SLIs

Step 1: define the two SLIs and the deadline

- **SLI 1, on-time completion.** Good: the invoice is posted correctly, or correctly routed to human review, within the deadline. Valid: every invoice that arrived, minus the exclusions in step 2.
- **SLI 2, correctness.** Good: a posted invoice whose key fields (invoice number, vendor, date, total) match the source. Valid: the audited sample, for example a random 5% of posted invoices checked by a human.
- **The deadline** is the "800 ms" of this project. We choose it from what the business needs, for example 4 hours from arrival. A longer deadline leaves more room for retries.
- **Why two:** a wrong value posted without an error is worse than a failure, and only an audited sample can see it.
- **Per-stage numbers** (download, extraction, LLM fallback, posting) are for debugging. They are not SLOs.

Step 2: decide the valid events

- **Exclude:** corrupt PDFs, duplicates, and agreed maintenance windows on a foreign site. These are not events at all.
- **Count internally:** a foreign site being down still hurts the customer, so it counts against the internal SLO. Tag every failure with a cause (ours, foreign source, foreign target, Content Understanding, LLM).
- **The SLA** can exclude foreign-site causes, if the customer agrees.

Step 3: calculate the SLIs

The SLI comes from one state table, one row per invoice, not from logs.

```
CREATE TABLE dbo.InvoiceRun (
    invoice_id        INT IDENTITY PRIMARY KEY,
    arrived_at        DATETIME2 NOT NULL,
    finished_at       DATETIME2 NULL,
    final_state       VARCHAR(20) NULL,  -- posted, human_review, failed
    exclusion_reason  VARCHAR(40) NULL,  -- corrupt_pdf, duplicate,
maintenance_window
    failure_cause     VARCHAR(40) NULL   -- ours, foreign_source,
foreign_target, content_understanding, llm
);

CREATE TABLE dbo.InvoiceAudit (
    invoice_id         INT PRIMARY KEY,
    audited_at         DATETIME2 NOT NULL,
    key_fields_correct BIT NOT NULL
);
```

SLI 1 over a rolling 30 days, with a 240-minute deadline:

```
DECLARE @deadline_min INT = 240;
DECLARE @now DATETIME2 = SYSUTCDATETIME();

SELECT
    SUM(CASE WHEN final_state IN ('posted', 'human_review')
              AND DATEDIFF(MINUTE, arrived_at, finished_at) <= @deadline_min
             THEN 1 ELSE 0 END)                  AS good,
    COUNT(*)                                     AS valid,
    100.0 * SUM(CASE WHEN final_state IN ('posted', 'human_review')
                      AND DATEDIFF(MINUTE, arrived_at, finished_at) <=
@deadline_min
                     THEN 1 ELSE 0 END)
          / NULLIF(COUNT(*), 0)                  AS sli_pct
FROM dbo.InvoiceRun
WHERE arrived_at >= DATEADD(DAY, -30, @now)
  AND exclusion_reason IS NULL
  AND (finished_at IS NOT NULL OR arrived_at <= DATEADD(MINUTE, -
@deadline_min, @now));
```

The last line leaves out invoices still in flight whose deadline has not passed yet. They are neither good nor bad. An invoice still unfinished after the deadline counts as bad.

SLI 2 over the same 30 days:

```
SELECT
    SUM(CAST(key_fields_correct AS INT))                    AS good,
    COUNT(*)                                                AS valid,
    100.0 * SUM(CAST(key_fields_correct AS INT)) / COUNT(*) AS sli_pct
FROM dbo.InvoiceAudit
WHERE audited_at >= DATEADD(DAY, -30, SYSUTCDATETIME());
```

A 5% sample is about 1,500 audits a month. That measures a 99% result to roughly plus or minus 0.5 percentage points, so treat small movements as noise.

Step 4: choose the SLO from a forecast

Start from today's reality, not from a wish. Run 2 to 4 weeks, then forecast the bad invoices per cause: touched share x failure rate after retries x 30,000.

Cause	Touches	Fails after retries	Bad invoices / 30 days
Source website download	100%	0.2%	60

Cause	Touches	Fails after retries	Bad invoices / 30 days
Content Understanding	100%	0.1%	30
LLM fallback	10%	1%	30
Target website automation	100%	0.6%	180
Our bugs and bad deploys	-	-	60
Total			**360 (1.2%)**

A 99% SLO allows only 300, so it doesn't fit yet. For the first version:

- **SLO 1:** 98.5% of valid invoices complete within 4 hours, over a rolling 30 days. Budget 1.5% = 450 bad invoices, about 15 a day.
- **Headroom:** 450 - 360 = 90 (20%), left for surprises.
- **Tighten to 99%** after the target-website automation is hardened. It is the biggest spender.
- **SLO 2:** 99% of audited invoices have correct key fields. In a sample of 1,500 that allows 15 wrong ones. It has its own budget.

Step 5: calculate the budget and what is left

- **Budget** = 100% - SLO = 1.5%.
- **Allowed bad** = valid invoices x 1.5%. With 30,000 that is 450.
- **Budget left** = 1 - (bad so far / allowed bad).

```
WITH m AS (
    SELECT
        SUM(CASE WHEN final_state IN ('posted', 'human_review')
                  AND DATEDIFF(MINUTE, arrived_at, finished_at) <= 240
                 THEN 1 ELSE 0 END) AS good,
        COUNT(*)                    AS valid
    FROM dbo.InvoiceRun
    WHERE arrived_at >= DATEADD(DAY, -30, SYSUTCDATETIME())
      AND exclusion_reason IS NULL
      AND (finished_at IS NOT NULL OR arrived_at <= DATEADD(MINUTE, -240,
SYSUTCDATETIME()))
)
SELECT
    valid,
    valid - good                            AS bad_so_far,
    valid * 0.015                           AS allowed_bad,   -- 1 - 98.5%
    1.0 - (valid - good) / (valid * 0.015)  AS budget_left    -- 0.35 means
35% left
FROM m;
```

The budget counts every bad invoice, whatever the cause: the target website, Content Understanding, the LLM or our own bug. The dependencies do not change the size of the budget. They spend from it. Their reliability matters when we pick the SLO, as in the forecast above.

To see who is spending it, group the bad invoices by cause and compare with the forecast:

```
SELECT failure_cause, COUNT(*) AS bad_invoices
FROM dbo.InvoiceRun
WHERE arrived_at >= DATEADD(DAY, -30, SYSUTCDATETIME())
  AND exclusion_reason IS NULL
  AND (final_state = 'failed'
       OR DATEDIFF(MINUTE, arrived_at, ISNULL(finished_at, SYSUTCDATETIME()))
> 240)
GROUP BY failure_cause
ORDER BY bad_invoices DESC;
```

Step 6: SLA

Looser than the SLO, for example 96% on time per calendar month (illustrative), with the excluded causes written down and credits if it is missed. Your legal or commercial team should word it.

Step 7: alerts and the policy

The same pairs as the burn-rate table, in invoices (budget 450):

Budget used	Window	Bad invoices	Action
2%	1 hour	9	page
5%	6 hours	about 23	page
10%	3 days	45	ticket

- At about 42 invoices an hour, one failure moves a short window a lot. Add a **stuck** page: nothing posted for 2 business hours.
- **Error budget policy:** when the budget is gone, pause new features and new vendor templates, and work on robustness (health checks, sturdier selectors, retries).

Step 8: review every month

Compare the actual spend by cause with the forecast. Tighten the SLO where the headroom allows, and move the biggest spender to the top of the backlog.

Recovery points for the foreign calls

A service can't be more reliable than the foreign sites it calls, unless a queue, retries and recovery points absorb the failures. From the retries and idempotency notes:

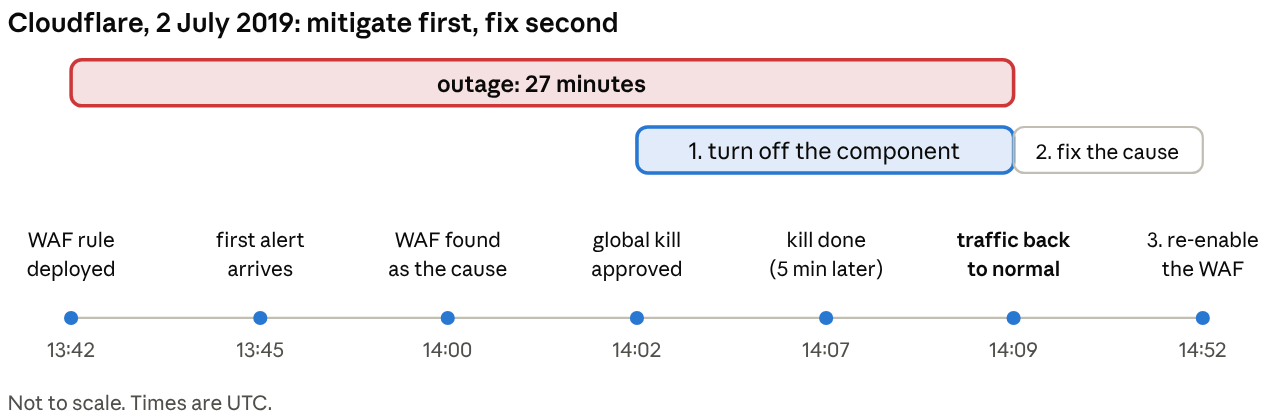
- Store a **recovery point** on each invoice row after every phase: `downloaded`, `extracted`, `llm_checked`, `posting_started`, `posted`. A retry resumes from the last one, so it doesn't re-download or re-extract.
- **Posting is the dangerous call.** There is no idempotency key on the target website. Write `posting_started` before the click. After a timeout, look the invoice up on the target site before retrying.
- **RPO and RTO** (my reading of recovery point): keep the state table durable so that `posted` is never lost. PDFs can be re-downloaded from the source. RTO should be well under the deadline, because a longer outage makes invoices late and spends budget.

Incident management

Why do we set up SLIs, SLOs, P99 and observability? They tell us that something is wrong. They only pay off if, when an outage happens, we can detect it, respond in an organised way, fix it, and learn from it. That process is **incident management**.

Case study: Cloudflare, 2 July 2019

- A new rule went into the WAF (web application firewall) and was deployed to the whole network at once. It contained a badly written regular expression that backtracked heavily and exhausted the CPU on every core serving HTTP traffic worldwide. The outage lasted **27 minutes**.
- **The first move was to mitigate, not to fix.** They used a **global kill switch** (Cloudflare's blog calls it a global terminate) to turn the WAF off worldwide. Traffic and CPU were back to normal at 14:09.
- Only then did they find and fix the cause. After tests, they re-enabled the WAF at 14:52.
- **The lesson: mitigate first, investigate second.** Restore the users first, then work out why.



Cloudflare outage timeline, 2 July 2019

- **Small clarification:** the mitigation was turning the WAF off, not rolling back the whole release. The rule change was rolled back and tested before the WAF came back on.
- **Declaring early worked:** the first alert came 3 minutes after the deploy, and SRE immediately declared a P0 incident and brought every relevant team online.
- **My addition, the 5 minutes between 14:02 and 14:07:** the kill was approved at 14:02 but done at 14:07, because the team's own login service (Access) was down and the bypass procedure was rarely used. A kill switch must be reachable during an outage,

and practised.

- **My addition, link to the SLO notes:** if this had been a full outage of a service with a 99.9% SLO, 27 minutes would use 62.5% of the 43.2-minute monthly budget.

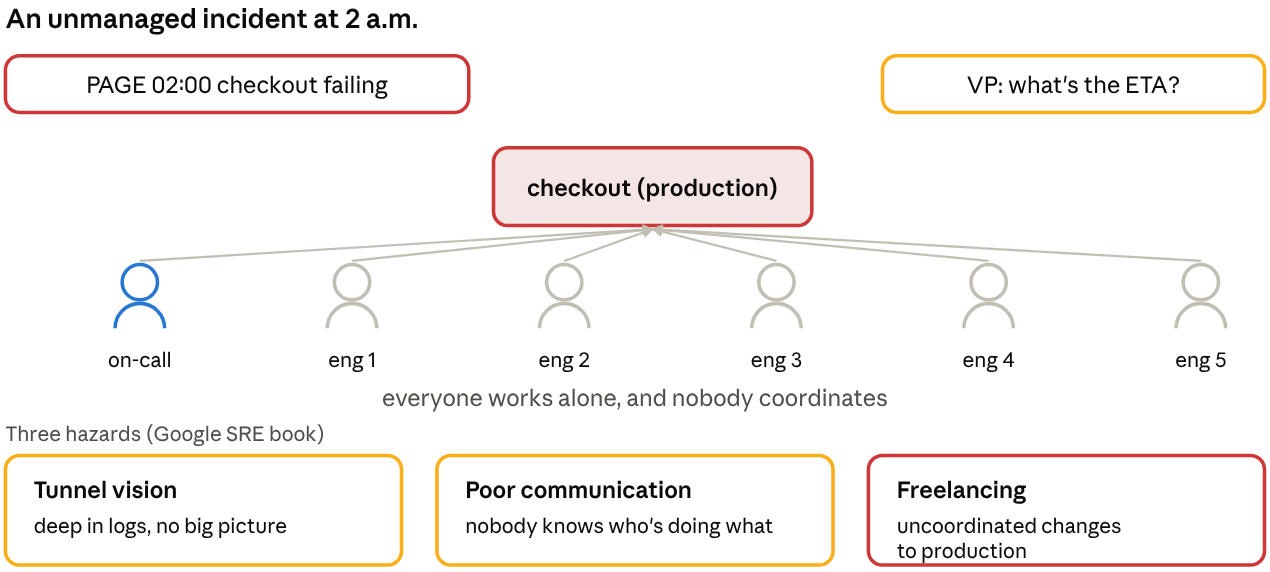
Source: Cloudflare, Details of the Cloudflare outage on July 2, 2019 (blog.cloudflare.com).

What goes wrong without a process

Take checkout failing at 02:00.

1. The page goes out and developers come online and start looking into it.
2. Every engineer works independently, and there is no coordination.
3. The director and the VP keep asking everyone for the ETA.
4. One engineer finds a fix and releases it on their own. Instead of a checkout issue, it becomes an outage of the whole service.

The cause: that decision was **freelancing**.



unmanaged incident and the three hazards

The Google SRE book names three hazards that make an incident worse:

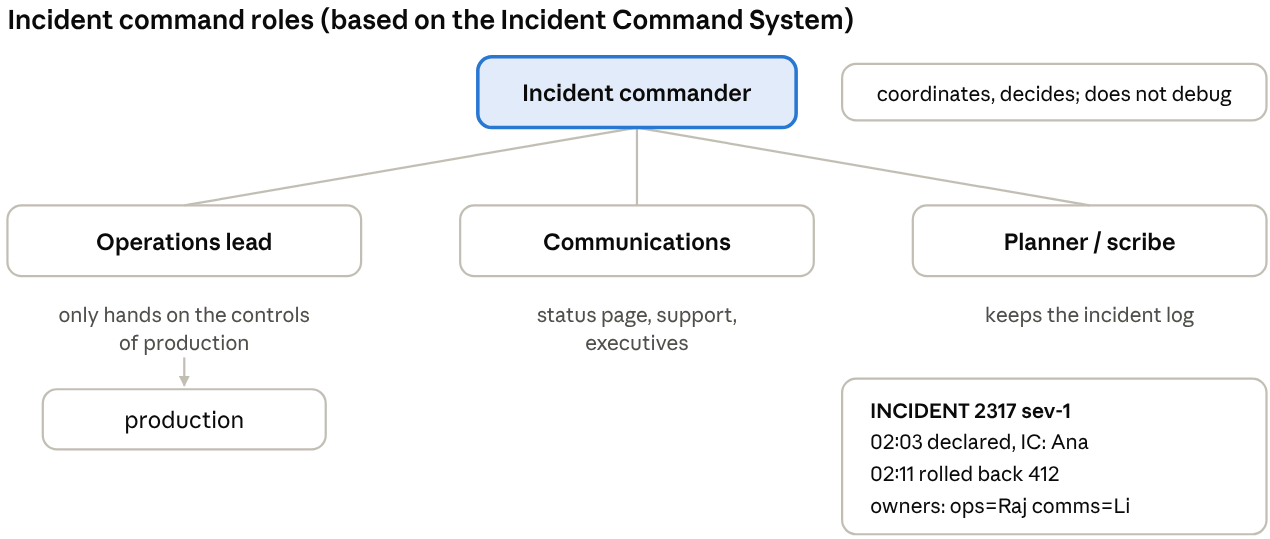
- **Tunnel vision:** someone looks too deep into the logs and loses the big picture.
- **Poor communication:** nobody knows who is doing what.
- **Freelancing:** uncoordinated changes to production.

The fix for all three is a process: **incident command**.

Incident command

Based on the Incident Command System: one person coordinates, and everyone else has a clear role.

- **Incident commander (IC):** coordinates and decides. **Does not debug.** Assigns the work.
- **Operations lead:** the only one with hands on the controls of production. Every change goes through this person, which stops freelancing.
- **Communications:** keeps the status page, support and executives informed, so engineers aren't interrupted by "what's the ETA?".
- **Planner / scribe:** keeps the incident log: the incident ID and severity, a timestamped timeline, and who owns what.



incident command roles and the scribe's log

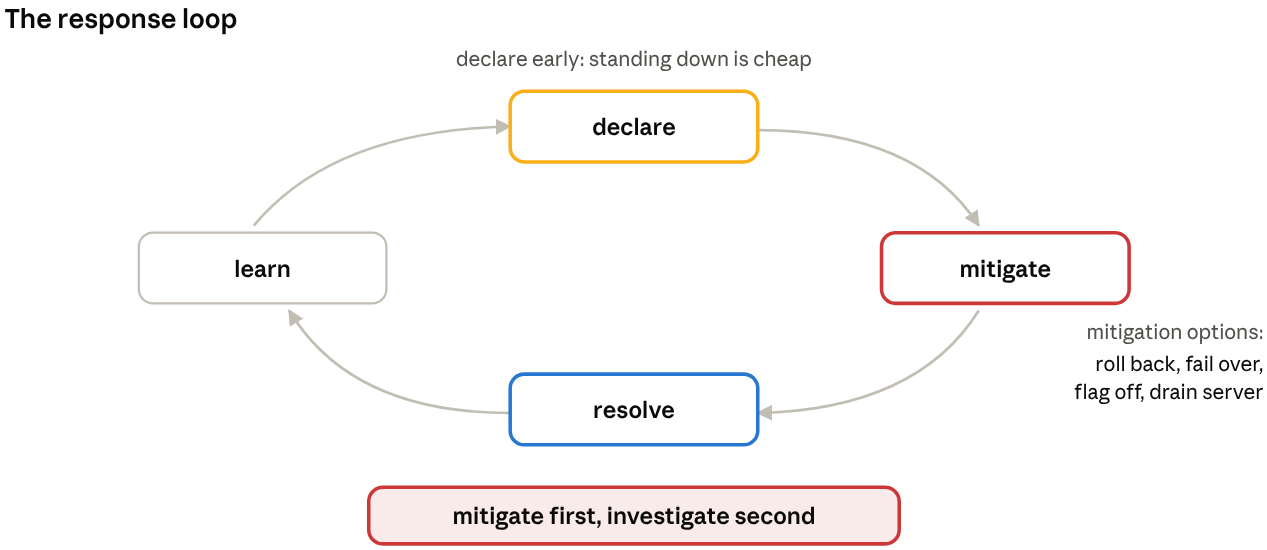
Who is the planner / scribe?

- It is a **role during the incident**, not a job title. The IC assigns it.
- Usually another engineer or technical person on the team who is not debugging, or a program or incident manager if the organisation has one. Either works.
- What matters is that they write things down quickly and accurately. Then the timeline for the postmortem is already done.
- **My addition (SRE book, as far as I know):** the planning role also handles longer-term items during the incident, like filing bugs and arranging handoffs.

- **My addition:** in a small team one person may hold more than one role, but the IC should not also be the one debugging or changing production.

The response loop

1. **Declare.** The incident commander declares the incident. Declare early: standing down is cheap.
2. **Mitigate.** Stop the user impact first with the quickest safe action: roll back, fail over, turn the flag off, or drain a server.
3. **Resolve.** Find and fix the real cause.
4. **Learn.** Run a blameless postmortem and share it with everyone.



the response loop

Mitigate first, investigate second. Someone digging into the cause while users are still failing is tunnel vision.

The Cloudflare case in this loop:

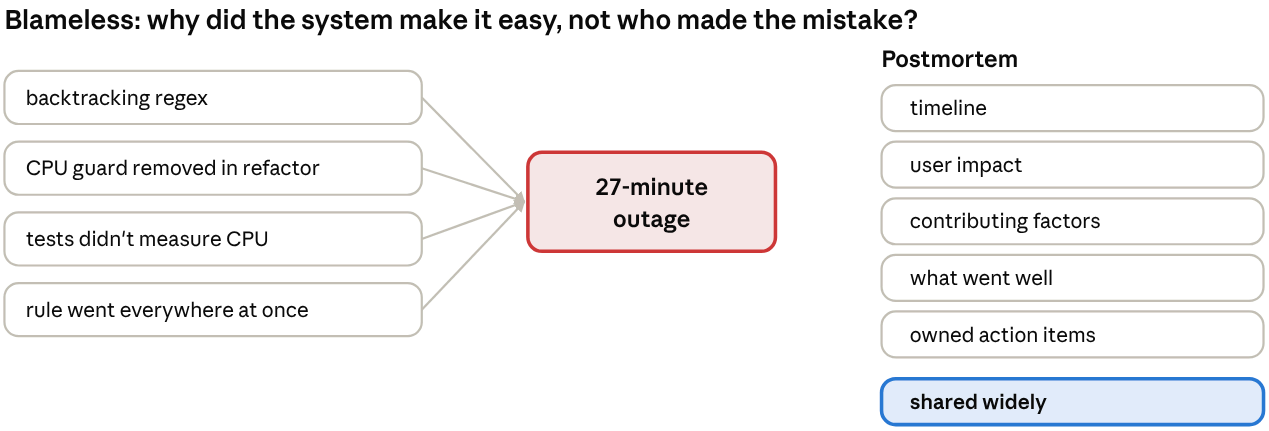
Step	What happened
Declare	SRE declared a P0 incident and brought every relevant team online
Mitigate	Global kill switch turned the WAF off, traffic normal at 14:09

Step	What happened
Resolve	Fixed and tested, WAF re-enabled at 14:52
Learn	Public postmortem published on 12 July

The kill switches, feature flags and gradual rollouts that make mitigation fast are covered under deployment safety.

Blameless postmortems

After the incident we ask **why did the system make it easy?**, not **who made the mistake?** The person who made the change is not the problem. A system that let one change take everything down is.



blameless postmortem, factors and contents

Contributing factors in the Cloudflare outage:

- A regular expression that backtracked heavily.
- A CPU protection removed by mistake during a refactor.
- Tests that didn't measure CPU use.
- A rule change that went to every location at once.

Cloudflare's blog lists more, for example a rollback plan that took too long, a slow first alert, a slow status page update, and difficulty reaching their own systems.

Action items, each with an owner:

Action item	Owner
Restore CPU protection	WAF team
Inspect all 3,868 rules	WAF team

My addition, also from the blog: add performance profiling to the test suite, and change the rule rollout to be staged.

A postmortem contains: a clear timeline, the user impact, the contributing factors, what went well, and owned action items. Then it is shared widely. An action item without an owner doesn't get done.

Recap

1. An unmanaged incident has three hazards: tunnel vision, poor communication, freelancing.
2. Roles: commander, operations, communications, scribe.
3. Declare early. Mitigate before you diagnose.
4. Blameless postmortems with owned actions.